Al-Mustafa University
Department of AI

Structured Programming C++

Part I: Theory Section

ECTS Credits: 8
Module Level: 1 | Semester: 2

Instructor
Dr. Husam Salah Mahdi
HOD AI Dep

First Year – Second Course
Academic Year: 2025–2026

Part I

Theory Section

Contents

CHAPTER 1

Functions in C++

1.1 Introduction to Functions

Definition: Function

A **function** is a self-contained set of statements designed to accomplish a particular task. Every C++ program contains at least one function — `main()` — and may define any number of additional functions.

In practice, large programs are difficult to write, read, and maintain as a single monolithic block of code. The solution is *modular programming*: dividing a program into smaller, manageable pieces called **functions**. Each function performs a specific, well-defined task, and the complete program is built by combining these functions together.

Advantages of Using Functions

1. **Easier to write:** Each function focuses on a single task, making the logic simpler to implement.
2. **Easier to read:** Well-named functions make the overall program flow self-documenting and understandable at a glance.
3. **Easier to debug:** Errors can be isolated to individual functions, reducing the search space during troubleshooting.
4. **Reusability:** A function can be called multiple times with different parameters, eliminating code duplication.
5. **Team development:** Different programmers can work on different functions simultaneously.

Modular Programming

Modular programming is a software design technique that decomposes a program into separate sub-programs (modules or functions). Each module is developed and tested independently, then integrated into the complete program. This approach reduces complexity and improves code quality.

1.2 Defining a Function

Every function definition in C++ consists of four parts: a **return type**, a **function name**, a **parameter list**, and a **function body**. The general form is:

```
return-type function-name(parameter-list)
{
    // body of the function
    statement1;
    statement2;
    ...
    return value;  // (if return-type is not void)
}
```

Components of a Function Definition

- **Return type:** The data type of the value returned by the function. Common types include `int`, `float`, `double`, `char`, and `void` (when the function returns nothing).
- **Function name:** A valid C++ identifier that describes the purpose of the function.
- **Parameter list:** A comma-separated list of typed variables that receive values from the caller. May be empty if the function requires no input.
- **Function body:** The block of statements enclosed in braces `{}` that define what the function does.

1.2.1 Function Prototype (Declaration)

A **function prototype** tells the compiler about a function's name, return type, and parameters *before* the function is fully defined. This allows `main()` (or any calling function) to appear before the function definition in the source file.

Definition: Function Prototype

A **function prototype** is a declaration of a function that specifies its return type, name, and parameter types, terminated by a semicolon. It does *not* include the function body. The general form is:

`return-type function-name(parameter-type-list);`

```
#include <iostream>
```

```cpp
using namespace std;

// Function prototype (declaration)
int add(int, int);

int main()
{
    int result = add(10, 20);
    cout << "Sum = " << result << endl;
    return 0;
}

// Function definition (after main)
int add(int a, int b)
{
    return a + b;
}
```

Output:

```
Sum = 30
```

Function Declaration Order

In C++, a function must be *declared* before it is called. You can either place the full function definition before `main()`, or provide a **function prototype** (declaration) before `main()` and define the function afterwards. If you call a function without a prior declaration, the compiler will report an error.

1.2.2 Function Return Types Summary

The following table summarises the most common return types used with functions:

Return Type	Example Signature	Description
`void`	`void greet()`	Performs an action; returns nothing.
`int`	`int square(int n)`	Returns an integer result.
`float`	`float aver(int a, int b)`	Returns a floating-point result.
`double`	`double area(double r)`	Returns a double-precision result.
`char`	`char grade(int mark)`	Returns a single character.
`bool`	`bool isEven(int n)`	Returns `true` or `false`.

1.2.3 Scope: Local vs. Global

Variables declared inside a function are **local** to that function — they exist only while the function is executing and cannot be accessed from outside. Variables declared outside all functions are **global** and can be accessed from any function in the program. A detailed discussion of scope follows in Chapter 2.

1.2.4 Example: A `void` Function

The simplest kind of function performs an action without returning a value. Its return type is `void`.

```
#include <iostream>
using namespace std;

// Function definition
void printMessage()
{
    cout << "Welcome to the world of C++ Functions!"
         << endl;
    cout << "Functions make programs modular and reusable."
         << endl;
}

int main()
{
    printMessage();   // Function call
    printMessage();   // Called again -- reusability in
       action
    return 0;
}
```

Output:

```
Welcome to the world of C++ Functions!
Functions make programs modular and reusable.
Welcome to the world of C++ Functions!
Functions make programs modular and reusable.
```

1.2.5 Example: A Function that Returns a Value

When a function computes and returns a result, its return type must match the type of the value returned.

```
#include <iostream>
using namespace std;
```

```cpp
// Function to find the maximum of two integers
int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}

int main()
{
    int x = 15, y = 27;
    int result = max(x, y);
    cout << "The maximum of " << x << " and " << y
         << " is: " << result << endl;
    return 0;
}
```

Output:

```
The maximum of 15 and 27 is: 27
```

1.3 The Return Statement

Definition: The return Statement

The `return` statement terminates execution of the current function and passes control back to the calling function. It has two forms:

- `return;` — used in `void` functions to exit early.
- `return expression;` — used in non-`void` functions to send a value back to the caller.

How return Works

When the `return` statement executes, the function stops immediately. Any statements after the `return` in the same block are *not* executed. In non-`void` functions, the returned expression is evaluated and its value is made available at the point of the function call.

1.3.1 Example: Calculating Squares of 1–10

The following program uses a function to compute the square of an integer and prints the squares of numbers from 1 to 10.

```cpp
#include <iostream>
using namespace std;

int square(int y)
{
    return y * y;
}

int main()
{
    cout << "Number\tSquare" << endl;
    cout << "------\t------" << endl;

    for (int x = 1; x <= 10; x++)
    {
        cout << x << "\t" << square(x) << endl;
    }

    return 0;
}
```

Output:

```
Number  Square
------  ------
1       1
2       4
3       9
4       16
5       25
6       36
7       49
8       64
9       81
10      100
```

1.3.2 Example: Calculating an Average

This function receives two integer values and returns their average as a floating-point number.

```cpp
#include <iostream>
using namespace std;

float aver(int x1, int x2)
{
    float result;
    result = (x1 + x2) / 2.0;
    return result;
}

int main()
{
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;

    float avg = aver(a, b);
    cout << "The average of " << a << " and " << b
         << " is: " << avg << endl;

    return 0;
}
```

Sample Output:

```
Enter two integers: 7 13
The average of 7 and 13 is: 10
```

Integer Division Pitfall

Note the use of `2.0` instead of `2` in the division. Dividing two integers in C++ produces an integer result (truncating the decimal part). Writing `2.0` forces floating-point division, ensuring an accurate average.

1.3.3 Example: Sum of Squares Series

The following program computes the sum of the series $1^2 + 2^2 + 3^2 + \cdots + n^2$ using a dedicated function.

```cpp
#include <iostream>
using namespace std;

int summation(int x)
{
```

```
    int sum = 0;
    for (int i = 1; i <= x; i++)
    {
        sum += i * i;
    }
    return sum;
}

int main()
{
    int n;
    cout << "Enter a positive integer n: ";
    cin >> n;

    int result = summation(n);
    cout << "The sum of squares from 1 to " << n
         << " is: " << result << endl;

    return 0;
}
```

Sample Output:

```
Enter a positive integer n: 5
The sum of squares from 1 to 5 is: 55
```

The computation: $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55$.

1.3.4 Example: Maximum of Three Integers

A function can call other functions. Here `max3()` finds the largest of three integers by calling the two-argument `max()` function twice.

```
#include <iostream>
using namespace std;

int max(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}

int max3(int a, int b, int c)
{
    return max(max(a, b), c);
```

```
}

int main()
{
    int x, y, z;
    cout << "Enter three integers: ";
    cin >> x >> y >> z;

    cout << "The largest value is: "
         << max3(x, y, z) << endl;

    return 0;
}
```

Sample Output:

```
Enter three integers: 14 7 21
The largest value is: 21
```

Composing Functions

Notice how `max3()` reuses `max()` rather than duplicating comparison logic. This demonstrates the principle of *function composition*: building complex behaviour by combining simpler functions.

1.3.5 Example: Logic Gates

Functions can also model logical operations. The following program implements the four basic logic gates — AND, OR, XOR, and NOT — as separate functions.

```
#include <iostream>
using namespace std;

bool AND(bool a, bool b) { return a && b; }
bool OR(bool a, bool b)  { return a || b; }
bool XOR(bool a, bool b) { return a != b; }
bool NOT(bool a)          { return !a;     }

int main()
{
    bool x = true, y = false;

    cout << "x = " << x << ", y = " << y << endl;
    cout << "AND(x,y) = " << AND(x, y) << endl;
```

```
    cout << "OR(x,y)  = " << OR(x, y)  << endl;
    cout << "XOR(x,y) = " << XOR(x, y) << endl;
    cout << "NOT(x)   = " << NOT(x)    << endl;
    cout << "NOT(y)   = " << NOT(y)    << endl;

    return 0;
}
```

Output:

```
x = 1, y = 0
AND(x,y) = 0
OR(x,y)  = 1
XOR(x,y) = 1
NOT(x)   = 0
NOT(y)   = 1
```

Logic Gate Truth Table

A	B	AND	OR	XOR	NOT A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

1.4 Passing Parameters

When a function is called, data can be communicated to it through **parameters** (also called **arguments**). C++ provides two fundamental mechanisms for passing parameters: *by value* and *by reference*.

1.4.1 Passing by Value

Definition: Pass by Value

When a parameter is passed **by value**, a *copy* of the argument's value is made and given to the function. Any changes the function makes to the parameter affect only the local copy; the original variable in the calling function remains unchanged.

All examples presented so far in this chapter use pass-by-value. Consider the `square()` function: when we call `square(x)`, the value of `x` is copied into the parameter `y`. Modifying `y` inside `square()` would have no effect on `x` in `main()`.

```cpp
#include <iostream>
using namespace std;

void tryToChange(int n)
{
    n = 999;    // Only changes the LOCAL copy
    cout << "Inside function: n = " << n << endl;
}

int main()
{
    int x = 5;
    cout << "Before call: x = " << x << endl;

    tryToChange(x);

    cout << "After call:  x = " << x << endl;
    return 0;
}
```

Output:

```
Before call: x = 5
Inside function: n = 999
After call:  x = 5
```

When to Use Pass by Value

Use pass by value when:

- The function only needs to *read* the argument.
- The argument is a small, inexpensive-to-copy type (e.g., `int`, `float`, `char`).
- You want to guarantee that the original variable is not modified.

1.4.2 Passing by Reference

Definition: Pass by Reference

When a parameter is passed **by reference**, the *address* of the original variable is passed to the function. The function receives a pointer (or reference) to the original data, so any changes made inside the function *directly affect* the original variable.

Passing by reference is useful when a function needs to modify variables in the calling scope, or when copying large data structures would be inefficient.

Advantages of Pass by Reference

- The function can **modify** the caller's variables directly.
- No copy is created, resulting in **higher execution speed** and **lower memory usage**, especially for large objects.
- A function can effectively "return" multiple values by modifying several referenced parameters.

Example: Swapping Two Values Using Pointers

The classic demonstration of pass-by-reference is a `swap` function that exchanges the values of two variables. Using pointers, the function receives the *addresses* of the variables and modifies them in place.

```
#include <iostream>
using namespace std;

void swap(int *a, int *b)
{
    int temp;
    temp = *a;    // Store the value at address a
    *a = *b;      // Copy value at address b into a
    *b = temp;    // Copy saved value into address b
}

int main()
{
    int x = 10, y = 20;

    cout << "Before swap:" << endl;
    cout << "  x = " << x << ", y = " << y << endl;
```

```
    swap(&x, &y);    // Pass addresses of x and y

    cout << "After swap:" << endl;
    cout << "  x = " << x << ", y = " << y << endl;

    return 0;
}
```

Output:

```
Before swap:
  x = 10, y = 20
After swap:
  x = 20, y = 10
```

Tracing the Swap Step by Step

The following table traces every statement inside `swap(&x, &y)` where initially `x = 10` and `y = 20`:

Step	Statement	temp	*a (x)	*b (y)
0	(entry)	—	10	20
1	`temp = *a;`	10	10	20
2	`*a = *b;`	10	20	20
3	`*b = temp;`	10	20	10

Because the function operates on the *actual memory locations* of `x` and `y`, the changes persist after the function returns.

Pass by Value Would Fail Here

If the swap function used pass-by-value (`void swap(int a, int b)`), only the *local copies* would be swapped. The original variables `x` and `y` in `main()` would remain unchanged. Always use pass-by-reference (via pointers or C++ references) when a function must modify its arguments.

Alternative: C++ Reference Syntax

C++ also supports pass-by-reference using the `&` (ampersand) notation in the parameter list, which provides a cleaner syntax than pointers:

```cpp
#include <iostream>
using namespace std;

void swap(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10, y = 20;

    cout << "Before swap: x = " << x
         << ", y = " << y << endl;

    swap(x, y);   // No need to pass addresses

    cout << "After swap:  x = " << x
         << ", y = " << y << endl;

    return 0;
}
```

Output:

```
Before swap: x = 10, y = 20
After swap:  x = 20, y = 10
```

Pointers vs. References

Both pointers and references achieve pass-by-reference behaviour. C++ references (`int &a`) are generally preferred because they are simpler, safer (cannot be null), and produce cleaner code. Pointers are still essential in many scenarios, such as dynamic memory allocation, arrays, and data structures.

1.5 Chapter Summary

Key Takeaways – Functions in C++

1. A **function** groups related statements into a reusable unit, promoting modularity, readability, and maintainability.
2. Every function has a **return type**, a **name**, a **parameter list**, and a **body**.
3. A **function prototype** declares a function before its definition, allowing flexible source-file organisation.
4. The `return` statement sends a value back to the caller and terminates the function.
5. **Pass by value** copies the argument; the original is safe from modification.
6. **Pass by reference** (via pointers or `&`) gives the function direct access to the caller's variable, enabling in-place modification.

CHAPTER 2

Advanced Function Concepts

2.1 Types of User-Defined Functions

User-defined functions in C++ can be classified into three categories based on whether they accept arguments and whether they return a value.

Classification of Functions

1. **No arguments, no return value** — The function neither receives data from the caller nor sends data back. It is declared as `void functionName(void)` or `void functionName()`.
2. **With arguments, no return value** — The function receives data through its parameters but does not return a result. It is declared as `void functionName(parameters)`.
3. **With arguments, with return value** — The function receives data and returns a computed result. It is declared as `type functionName(parameters)`.

2.1.1 Function Types Summary Table

Type	Arguments	Return	Example Signature	Use Case
1	None	None	`void greet()`	Print a menu / banner
2	Yes	None	`void printSquare(int n)`	Display formatted output
3	Yes	Yes	`float calcSquare(float n)`	Compute & return a result

2.1.2 Type 1: No Arguments, No Return Value

```cpp
#include <iostream>
using namespace std;

void greet()
{
    cout << "Hello! Welcome to C++ programming."
         << endl;
    cout << "Let's learn about functions!" << endl;
}

int main()
{
    greet();    // No arguments, no return value
    return 0;
}
```

Output:

```
Hello! Welcome to C++ programming.
Let's learn about functions!
```

2.1.3 Type 2: With Arguments, No Return Value

```cpp
#include <iostream>
using namespace std;

void printSquare(int n)
{
    cout << "The square of " << n
         << " is: " << n * n << endl;
}

int main()
{
    int x = 7;
    printSquare(x);    // Argument passed, no return
    printSquare(12);   // Can also pass a literal
    return 0;
}
```

Output:

```
The square of 7 is: 49
The square of 12 is: 144
```

2.1.4 Type 3: With Arguments, With Return Value

```cpp
#include <iostream>
using namespace std;

float calcSquare(float n)
{
    return n * n;
}

int main()
{
    float x = 3.5;
    float result = calcSquare(x);
    cout << "The square of " << x
         << " is: " << result << endl;

    // Returned value used directly in an expression
    cout << "Twice the square: "
         << 2 * calcSquare(x) << endl;
    return 0;
}
```

Output:

```
The square of 3.5 is: 12.25
Twice the square: 24.5
```

Choosing the Right Type

- Use **Type 1** for standalone actions such as printing a menu or a welcome message.
- Use **Type 2** when the function needs input but only performs an action (e.g., displaying output) without producing a result.
- Use **Type 3** when the function computes and returns a value that the caller needs for further processing.

2.1.5 Side-by-Side Comparison: `void` vs. Return Value

Consider two approaches to computing the square of a number:

```cpp
#include <iostream>
using namespace std;
```

```cpp
// Approach A: void -- prints directly
void squareVoid(float n)
{
    cout << "Square (void): " << n * n << endl;
}

// Approach B: returns the result
float squareReturn(float n)
{
    return n * n;
}

int main()
{
    float num = 4.0;

    // Approach A: printed inside the function
    squareVoid(num);

    // Approach B: returned and used further
    float res = squareReturn(num);
    cout << "Square (return): " << res << endl;

    // Returned value in an expression
    cout << "Twice the square: "
         << 2 * squareReturn(num) << endl;

    return 0;
}
```

Output:

```
Square (void): 16
Square (return): 16
Twice the square: 32
```

Return Values Enable Composition

Functions that *return* values are more flexible because their results can be stored in variables, passed to other functions, or used in expressions. The `void` approach is suitable only when the sole purpose of the function is to produce a side effect (such as printing).

2.2 Actual and Formal Arguments

When working with functions, it is essential to distinguish between **actual arguments** and **formal arguments**.

Definition: Actual Arguments (Actual Parameters)

Actual arguments are the variables, constants, or expressions specified in the *function call*. They represent the real data that is passed to the function when it is invoked.

Definition: Formal Arguments (Formal Parameters)

Formal arguments (also called *dummy variables* or *parametric variables*) are the variables declared in the *function definition* header. They act as placeholders that receive the values of the actual arguments when the function is called.

The following example illustrates the distinction clearly:

```
#include <iostream>
using namespace std;

// Function prototype
int add(int a, int b);  // a, b = formal arguments

int main()
{
    int x = 5, y = 3;

    // x and y are ACTUAL arguments
    int sum = add(x, y);
    cout << "Sum = " << sum << endl;

    // Constants can also be actual arguments
    cout << "Sum = " << add(10, 20) << endl;

    return 0;
}

// Function definition
int add(int a, int b)   // a, b = FORMAL arguments
{
    // a receives value of x, b receives value of y
    return a + b;
```

```
}
```

Output:

```
Sum = 8
Sum = 30
```

Mapping Between Actual and Formal Arguments

When `add(x, y)` is called:

- The actual argument `x` (value 5) is copied into the formal argument `a`.
- The actual argument `y` (value 3) is copied into the formal argument `b`.

The names of actual and formal arguments need not match. What matters is the **order**, **number**, and **type** of the arguments — these must be consistent between the call and the definition.

Property	Actual Arguments	Formal Arguments
Appear in	Function *call*	Function *definition*
Also called	Real parameters	Dummy / parametric variables
Examples	`x`, `y`, `10`	`a`, `b`
Matching rule	Must agree in order, number, and type	

Common Mistakes with Arguments

- **Mismatched types:** Passing a `float` where an `int` is expected may cause implicit truncation.
- **Wrong number of arguments:** The number of actual arguments must match the number of formal parameters (unless default values are provided).
- **Wrong order:** Arguments are matched by *position*, not by name. Swapping the order can produce incorrect results.

2.3 Local and Global Variables

The **scope** of a variable determines where in a program it can be accessed. C++ distinguishes between local and global scope.

2.3.1 Local Variables

Definition: Local Variable

A **local variable** is a variable defined *inside* a function or a block (enclosed in `{}`). It exists only during the execution of that block and is destroyed when the block ends. Local variables cannot be accessed from outside their enclosing block.

```
#include <iostream>
using namespace std;

void func(int i, int j)
{
    int k, m;          // k and m are LOCAL to func()
    k = i + j;
    m = i * j;
    cout << "k = " << k << ", m = " << m << endl;
}

int main()
{
    int a = 4, b = 5; // a and b are LOCAL to main()
    func(a, b);

    // cout << k;  // ERROR: k is not accessible here
    return 0;
}
```

Output:

```
k = 9, m = 20
```

Key Properties of Local Variables

- Local variables **belong to the block** in which they are declared.
- Variables with the **same name** declared in different functions (or blocks) are **completely independent entities**, each with its own memory location.
- Local variables are allocated on the **stack** and are automatically destroyed when the function returns.
- Function parameters (`i` and `j` above) are also treated as local variables within the function.

2.3.2 Global Variables

Definition: Global Variable

A **global variable** is a variable defined *outside* all functions, typically at the top of the source file. It retains the same data type and name throughout the entire program and can be accessed from any function.

```cpp
#include <iostream>
using namespace std;

// Global variables -- accessible from any function
int x;
int y = 5;

void sum(void)
{
    // Reads and modifies global variables
    int s = x + y;
    cout << "x = " << x << ", y = " << y << endl;
    cout << "Sum = " << s << endl;
}

int main()
{
    x = 10;       // Assign to global x
    sum();

    x = 100;
    y = 200;
    sum();         // Globals have new values now

    return 0;
}
```

Output:

```
x = 10, y = 5
Sum = 15
x = 100, y = 200
Sum = 300
```

Global Variable Initialization

Global variables in C++ are automatically **zero-initialized** if no explicit initial value is provided. In the example above, `int x;` starts at 0, while `int y = 5;` starts at 5.

Here is a second example showing how multiple functions share global data:

```cpp
#include <iostream>
using namespace std;

// Global variables
int x, y;

void readValues()
{
    cout << "Enter two integers: ";
    cin >> x >> y;    // Modifying global variables
}

void printSum()
{
    int sum = x + y;  // Reading global variables
    cout << "The sum of " << x << " and " << y
         << " is: " << sum << endl;
}

int main()
{
    readValues();
    printSum();
    return 0;
}
```

Sample Output:

```
Enter two integers: 12 8
The sum of 12 and 8 is: 20
```

Use Global Variables Sparingly

While global variables provide a convenient way to share data between functions, they have significant drawbacks:

- Any function can modify a global variable, making it difficult to track where

changes occur.
- Global variables increase **coupling** between functions, reducing modularity.
- They can lead to subtle bugs when two functions unintentionally modify the same variable.

Prefer passing data through **parameters and return values** whenever possible. Reserve global variables for data that genuinely needs to be shared across many functions.

Scope Summary

Property	Local Variable	Global Variable
Declared in	Inside a function/block	Outside all functions
Accessible from	Only its own block	Any function
Lifetime	Block execution	Entire program
Default value	Undefined (garbage)	Zero-initialized
Storage	Stack	Data segment

2.4 Recursive Functions

Definition: Recursion

Recursion is a programming technique in which a function calls *itself*, either directly or indirectly. A function that employs recursion is called a **recursive function**.

Recursion is a powerful tool that is particularly useful for problems that can be broken down into smaller instances of the same problem, such as traversing **linked lists**, searching **trees**, and computing mathematical sequences.

Two Essential Components of a Recursive Function

Every recursive function must have:
1. **Base case:** A condition under which the function does *not* call itself, preventing infinite recursion.
2. **Recursive case:** The function calls itself with a modified argument that

moves toward the base case.
Without a proper base case, the function will call itself indefinitely, eventually causing a **stack overflow** error.

Recursion vs. Iteration

- A **normal (iterative) function** uses loops (`for`, `while`) to repeat operations.
- A **recursive function** achieves repetition by calling itself with progressively simpler inputs.
- Any problem solvable with recursion can also be solved iteratively, and vice versa. The choice depends on clarity and efficiency.

2.4.1 Example: Recursive Sum $1 + 2 + \cdots + n$

The sum of the first n positive integers can be defined recursively as:

$$\text{sum}(n) = \begin{cases} 0 & \text{if } n = 0 \quad \text{(base case)} \\ n + \text{sum}(n-1) & \text{if } n > 0 \quad \text{(recursive case)} \end{cases}$$

```cpp
#include <iostream>
using namespace std;

int sum(int n)
{
    if (n == 0)
        return 0;            // Base case
    else
        return n + sum(n - 1);  // Recursive case
}

int main()
{
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;

    cout << "Sum from 1 to " << n
         << " = " << sum(n) << endl;

    return 0;
}
```

Sample Output:

```
Enter a positive integer: 5
Sum from 1 to 5 = 15
```

Trace Table for `sum(5)`

Phase 1 — Recursive descent (pushing calls onto the stack):

Call	Function Call	Condition	Action
1	`sum(5)`	$5 \neq 0$	returns $5 +$ `sum`(4) — waits
2	`sum(4)`	$4 \neq 0$	returns $4 +$ `sum`(3) — waits
3	`sum(3)`	$3 \neq 0$	returns $3 +$ `sum`(2) — waits
4	`sum(2)`	$2 \neq 0$	returns $2 +$ `sum`(1) — waits
5	`sum(1)`	$1 \neq 0$	returns $1 +$ `sum`(0) — waits
6	`sum(0)`	$0 = 0$	returns 0 (base case reached)

Phase 2 — Stack unwinding (returning values):

Return	Returning From	Computation	Value
6	`sum(0)`	base case	0
5	`sum(1)`	$1 + 0$	1
4	`sum(2)`	$2 + 1$	3
3	`sum(3)`	$3 + 3$	6
2	`sum(4)`	$4 + 6$	10
1	`sum(5)`	$5 + 10$	**15**

Verification using the closed-form formula:

$$\texttt{sum}(5) = \frac{5 \times 6}{2} = \boxed{15}$$

2.4.2 Example: Recursive Factorial

The factorial of a non-negative integer n is defined as:

$$n! = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \quad \text{(base case)} \\ n \times (n-1)! & \text{if } n > 1 \quad \text{(recursive case)} \end{cases}$$

```cpp
#include <iostream>
using namespace std;
```

```cpp
long long factorial(int n)
{
    if (n <= 1)
        return 1;            // Base case: 0!=1!=1
    else
        return n * factorial(n - 1); // Recursive
}

int main()
{
    int n;
    cout << "Enter a non-negative integer: ";
    cin >> n;

    cout << n << "! = " << factorial(n) << endl;

    return 0;
}
```

Sample Output:

```
Enter a non-negative integer: 6
6! = 720
```

Trace Table for `factorial(6)`

Phase 1 — Recursive descent:

Call	Function Call	Condition	Action
1	`factorial(6)`	$6 > 1$	returns $6 \times$ `factorial`(5)
2	`factorial(5)`	$5 > 1$	returns $5 \times$ `factorial`(4)
3	`factorial(4)`	$4 > 1$	returns $4 \times$ `factorial`(3)
4	`factorial(3)`	$3 > 1$	returns $3 \times$ `factorial`(2)
5	`factorial(2)`	$2 > 1$	returns $2 \times$ `factorial`(1)
6	`factorial(1)`	$1 \leq 1$	returns 1 (base case)

Phase 2 — Stack unwinding:

Return	Returning From	Computation	Value
6	`factorial(1)`	base case	1
5	`factorial(2)`	2×1	2
4	`factorial(3)`	3×2	6
3	`factorial(4)`	4×6	24
2	`factorial(5)`	5×24	120
1	`factorial(6)`	6×120	**720**

Verification:

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = \boxed{720}$$

Stack Overflow Risk

Each recursive call adds a new frame to the program's **call stack**. For very large inputs (e.g., `factorial(100000)`), the stack may run out of memory, causing a *stack overflow* crash. Always ensure that:

- The base case is reachable for all valid inputs.
- Each recursive call moves closer to the base case.
- The expected recursion depth is within practical limits.

When to Use Recursion

Recursion is most natural for problems with a *recursive structure*:

- **Mathematical definitions:** Factorial, Fibonacci, power functions.
- **Data structures:** Traversing linked lists, binary trees, and graphs.
- **Divide-and-conquer algorithms:** Merge sort, quick sort, binary search.

For simple sequential tasks, an iterative solution (`for`/`while` loop) is usually more efficient and easier to understand.

2.5 Series Computation Using Functions

A common programming exercise is to compute the value of a mathematical series term by term. The following example computes the series:

$$S = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots \pm \frac{x^n}{n!}$$

where n is an odd positive integer. This is related to the Taylor series expansion of $\sin(x)$.

```cpp
#include <iostream>
#include <cmath>
using namespace std;

// Compute factorial of n
long long fact(int n)
{
    long long result = 1;
    for (int i = 2; i <= n; i++)
        result *= i;
    return result;
}

// Compute x raised to the power p
double power(double x, int p)
{
    double result = 1.0;
    for (int i = 0; i < p; i++)
        result *= x;
    return result;
}

int main()
{
    double x;
    int n;

    cout << "Enter value of x (radians): ";
    cin >> x;
    cout << "Enter number of terms n: ";
    cin >> n;

    double sum = 0.0;
    int sign = 1;

    for (int i = 0; i < n; i++)
    {
        int exp = 2 * i + 1;  // 1, 3, 5, 7, ...
        sum += sign * power(x, exp) / fact(exp);
        sign *= -1;             // Alternate sign
    }

    cout << "Series sum  = " << sum << endl;
    cout << "sin(" << x << ") = " << sin(x) << endl;

    return 0;
}
```

Sample Output:

```
Enter value of x (radians): 1.0
Enter number of terms n: 5
Series sum  = 0.841471
sin(1.0) = 0.841471
```

Understanding the Series Terms

For $x = 1.0$ and 5 terms, the computation proceeds as follows:

Term	Exponent	Sign	Expression	Value
1	1	+	$\frac{1^1}{1!} = \frac{1}{1}$	1.000000
2	3	−	$\frac{1^3}{3!} = \frac{1}{6}$	−0.166667
3	5	+	$\frac{1^5}{5!} = \frac{1}{120}$	0.008333
4	7	−	$\frac{1^7}{7!} = \frac{1}{5040}$	−0.000198
5	9	+	$\frac{1^9}{9!} = \frac{1}{362880}$	0.000003
			Sum	**0.841471**

2.6 Chapter Summary

Key Takeaways – Advanced Function Concepts

1. User-defined functions fall into **three categories**: no args / no return, args / no return, and args / with return.
2. **Actual arguments** are the values passed in a function call; **formal arguments** are the parameters in the function definition that receive those values.
3. **Local variables** exist only within their enclosing block; **global variables** are accessible throughout the program.
4. **Recursive functions** call themselves and must have both a *base case* and a *recursive case*.
5. Always trace recursive calls using a **descent / unwind table** to verify correctness and understand the call stack.
6. Functions enable **series computation** by encapsulating repetitive mathematical operations (factorial, power) into reusable units.

CHAPTER 3

One-Dimensional Arrays

3.1 Introduction to Arrays

In many programming situations, we need to store and manipulate collections of related data items—for instance, the grades of all students in a class or the daily temperatures over a month. Declaring a separate variable for each value quickly becomes impractical. *Arrays* provide an elegant solution.

Definition: Array

An **array** is a consecutive group of homogeneous memory locations that all share the same name and the same data type. Each individual location is called an **element** and is accessed through an **index** (subscript).

Key characteristics of arrays:

- All elements occupy *contiguous* (adjacent) memory locations.
- Every element is of the *same* data type.
- Elements are referenced by combining the array name with a non-negative integer index enclosed in square brackets.
- Arrays can hold simple built-in types (`int`, `float`, `double`, `char`) as well as user-defined types.

Why Use Arrays?

Without arrays, storing ten exam scores would require ten separate variables (`score0`, `score1`, ..., `score9`). With an array we write `int score[10];` and access each value through its index—much cleaner, and far easier to process with loops.

3.2 Declaring One-Dimensional Arrays

A one-dimensional array is the simplest form of an array. Its declaration follows the general syntax:

Array Declaration Syntax

```
data-type  Array-name[size];
```

- `data-type` — the type of each element (e.g. `int`, `float`, `char`).
- `Array-name` — a valid C++ identifier.
- `size` — a positive integer constant specifying the number of elements.

Examples.

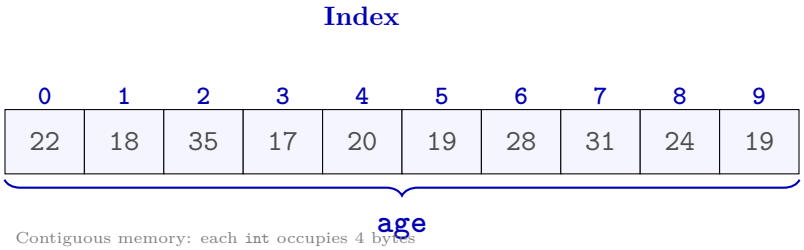
```
int   age[10];     // 10 integers
int   num[30];     // 30 integers
float degree[5];   // 5 floating-point numbers
char  a[15];       // 15 characters
```

Array Size Must Be a Constant

In standard C++, the size of an array must be a compile-time constant expression. Using a variable as the size (variable-length arrays) is not part of the C++ standard, although some compilers accept it as an extension.

Visualising an Array in Memory

The following diagram illustrates an integer array `age` of size 10. Each box represents one element; the numbers above the boxes are the indices (0 through 9).



Memory Layout Detail

When you declare `int age[10];`, the compiler allocates $10 \times$ `sizeof(int)` $= 10 \times 4 = 40$ bytes of contiguous memory. The address of element `age[i]` is `base_address + i*sizeof(int)`. This is why array access by index is an $O(1)$ operation.

3.3 Initialising Array Elements

There are several ways to assign initial values to the elements of an array.

3.3.1 Individual Element Initialisation

Each element can be assigned a value independently:

```
age[0] = 18;
age[9] = 19;
```

3.3.2 Initialiser-List Initialisation

All elements can be initialised at the point of declaration using an *initialiser list*:

```
int age[9] = {18, 17, 18, 18, 19, 20, 17, 18, 19};
```

3.3.3 Auto-Sized Initialisation

When an initialiser list is provided, the compiler can deduce the size of the array automatically:

```
int x[] = {12, 3, 5, 0, 11, 7, 30, 100, 22};   // size = 9
```

3.3.4 Partial Initialisation with Explicit Size

If fewer initialisers are provided than the declared size, the remaining elements are automatically initialised to zero:

```
int y[10] = {8, 10, 13, 15, 0, 1, 17, 22};
// y[8] and y[9] are automatically set to 0
```

Zero-Initialisation Shortcut

To set every element of an array to zero, write:

```
int arr[100] = {0};
```

The first element is explicitly set to `0`, and the rest are zero-initialised by the partial-initialisation rule.

The following table summarises all four initialisation methods:

Method	Syntax Example
Individual assignment	`age[0] = 18; age[1] = 17;`
Full initialiser list	`int a[3] = {10, 20, 30};`
Auto-sized	`int a[] = {10, 20, 30};`
Partial (rest = 0)	`int a[5] = {10, 20};`

3.4 Accessing Array Elements

Individual elements are accessed by appending the index in square brackets to the array name. The index of the *first* element is always `0`; the index of the *last* element is `size - 1`.

```
int x, y;
x = num[0];    // copy the first element into x
y = num[9];    // copy the tenth element into y
```

Array elements can participate in any expression, just like ordinary variables:

```
cout << num[0] + num[9];      // print the sum
num[0] = num[1] + num[2];    // assign the sum
num[7] += 3;                  // increment by 3
```

Index Out of Bounds

C++ does **not** perform automatic bounds checking on array indices. If you access `num[10]` in a 10-element array (valid indices 0–9), the behaviour is *undefined*—your program may crash, produce garbage values, or corrupt other data.

3.5 Reading, Writing, and Processing Arrays

Because arrays work hand-in-hand with loops, nearly all array processing uses a `for` loop whose counter variable serves as the index.

3.5.1 Writing (Printing) a Single Element

```
cout << num[4];    // print the element at index 4
```

3.5.2 Reading an Array from the User

```
#include <iostream>
using namespace std;

int main() {
    int num[10];

    cout << "Enter 10 integers:" << endl;
    for (int i = 0; i < 10; i++) {
        cin >> num[i];
    }

    return 0;
}
```

3.5.3 Printing an Array in Forward Order

```
cout << "Array elements (forward):" << endl;
for (int i = 0; i < 10; i++) {
    cout << num[i] << " ";
}
cout << endl;
```

3.5.4 Printing an Array in Reverse Order

```
cout << "Array elements (reverse):" << endl;
for (int i = 9; i >= 0; i--) {
    cout << num[i] << " ";
}
cout << endl;
```

3.5.5 Computing the Sum of All Elements

```
int sum = 0;
for (int i = 0; i < 10; i++) {
    sum += num[i];
}
cout << "Sum = " << sum << endl;
```

3.6 Practical Examples

3.6.1 Example 1: Accessing Specific Elements

Problem. Given the array `double distance[] = {23.14, 70.52, 104.08, 468.78, 6.28}`, display the 2nd and 5th elements.

```
#include <iostream>
using namespace std;

int main() {
    double distance[] = {23.14, 70.52, 104.08,
                         468.78, 6.28};

    cout << "2nd element: " << distance[1] << endl;
    cout << "5th element: " << distance[4] << endl;

    return 0;
}
```

Output:

```
2nd element: 70.52
5th element: 6.28
```

Indexing Starts at Zero

The "2nd element" is at index 1, and the "5th element" is at index 4. Always remember: the k-th element is at index $k - 1$.

3.6.2 Example 2: Read and Print in Reverse

Problem. Read 5 numbers from the user and print them in reverse order.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[5];

    cout << "Enter 5 numbers:" << endl;
    for (int i = 0; i < 5; i++) {
        cin >> arr[i];
    }

    cout << "Numbers in reverse order:" << endl;
    for (int i = 4; i >= 0; i--) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Sample run:

```
Enter 5 numbers:
10 20 30 40 50
Numbers in reverse order:
50 40 30 20 10
```

3.6.3 Example 3: Summation of Array Elements

Problem. Read 10 integers into an array and compute their summation.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[10];
    int sum = 0;

    cout << "Enter 10 integers:" << endl;
    for (int i = 0; i < 10; i++) {
        cin >> arr[i];
    }

    for (int i = 0; i < 10; i++) {
        sum += arr[i];
    }
```

```
    cout << "Summation = " << sum << endl;

    return 0;
}
```

Sample run:

```
Enter 10 integers:
1 2 3 4 5 6 7 8 9 10
Summation = 55
```

3.6.4 Example 4: Finding the Minimum Value

Problem. Read 8 numbers into an array and find the minimum value.

```
#include <iostream>
using namespace std;

int main() {
    int arr[8];

    cout << "Enter 8 numbers:" << endl;
    for (int i = 0; i < 8; i++) {
        cin >> arr[i];
    }

    int min = arr[0];
    for (int i = 1; i < 8; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }

    cout << "Minimum value = " << min << endl;

    return 0;
}
```

Sample run:

```
Enter 8 numbers:
34 12 56 7 89 23 45 11
Minimum value = 7
```

Finding the Minimum — Strategy

1. Assume the first element is the minimum.
2. Compare it with every subsequent element.
3. Whenever a smaller element is found, update the minimum.
4. After the loop completes, the variable holds the true minimum.

3.6.5 Example 5: Days in Each Month

Problem. Use an array to store the number of days in each month (non-leap year) and display them.

```
#include <iostream>
using namespace std;

int main() {
    int days[] = {31, 28, 31, 30, 31, 30,
                  31, 31, 30, 31, 30, 31};

    string months[] = {
        "January",   "February", "March",
        "April",     "May",      "June",
        "July",      "August",   "September",
        "October",   "November", "December"
    };

    for (int i = 0; i < 12; i++) {
        cout << months[i] << " has "
             << days[i] << " days." << endl;
    }

    return 0;
}
```

Output (first four lines shown):

```
January has 31 days.
February has 28 days.
March has 31 days.
April has 30 days.
...
```

Parallel Arrays

Using two arrays of the same size—one for month names and one for day counts—is called the **parallel array** technique. The element at index `i` in one array corresponds to the element at the same index in the other.

3.6.6 Example 6: Searching for a Value (Using a Function)

Problem. Write a function that searches for a value `X` in an array and returns its index. If the value is not found, return `-1`.

```
#include <iostream>
using namespace std;

int search(int arr[], int size, int x) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == x) {
            return i;    // found: return the index
        }
    }
    return -1;           // not found
}

int main() {
    int arr[] = {10, 25, 33, 47, 52, 68, 71, 89};
    int size = 8;
    int x;

    cout << "Enter a value to search for: ";
    cin >> x;

    int index = search(arr, size, x);

    if (index != -1) {
        cout << x << " found at index "
             << index << endl;
    } else {
        cout << x << " not found in the array."
             << endl;
    }

    return 0;
}
```

Sample run 1:

```
Enter a value to search for: 47
47 found at index 3
```

Sample run 2:

```
Enter a value to search for: 99
99 not found in the array.
```

Linear Search

The algorithm shown above is called **linear search** (or *sequential search*). It examines each element one by one from the beginning of the array until the target value is found or the end of the array is reached. Its time complexity is $O(n)$.

3.6.7 Example 7: Splitting Odd and Even Numbers

Problem. Read 10 integers into an array, then split them into two separate arrays: one for odd numbers and one for even numbers.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[10];
    int odd[10], even[10];
    int oddCount = 0, evenCount = 0;

    cout << "Enter 10 integers:" << endl;
    for (int i = 0; i < 10; i++) {
        cin >> arr[i];
    }

    for (int i = 0; i < 10; i++) {
        if (arr[i] % 2 == 0) {
            even[evenCount] = arr[i];
            evenCount++;
        } else {
            odd[oddCount] = arr[i];
            oddCount++;
        }
    }

    cout << "Even numbers: ";
    for (int i = 0; i < evenCount; i++) {
        cout << even[i] << " ";
    }
```

```
    cout << endl;

    cout << "Odd numbers: ";
    for (int i = 0; i < oddCount; i++) {
        cout << odd[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Sample run:

```
Enter 10 integers:
3 8 15 22 7 40 11 6 9 18
Even numbers: 8 22 40 6 18
Odd numbers: 3 15 7 11 9
```

Tracking Array Sizes with Counters

When the number of elements to be stored in an array is not known in advance, maintain a separate counter variable (e.g. `oddCount`, `evenCount`) that tracks how many elements have been inserted so far. This counter also serves as the index for the next insertion.

CHAPTER 4

Two-Dimensional Arrays

4.1 Introduction to 2D Arrays

A one-dimensional array stores a single list of values. In many real-world situations, however, data is naturally organised in a tabular (row-and-column) format—exam scores for several students across multiple subjects, pixel intensities in an image, or entries in a mathematical matrix. For such cases C++ provides *two-dimensional arrays*.

Definition: Two-Dimensional Array

A **two-dimensional (2D) array** is an array of arrays. It is a collection of elements arranged in rows and columns, where each element is accessed using *two* indices: a **row index** and a **column index**.

Declaration Syntax

```
data-type  Array-name[Row-size][Col-size];
```

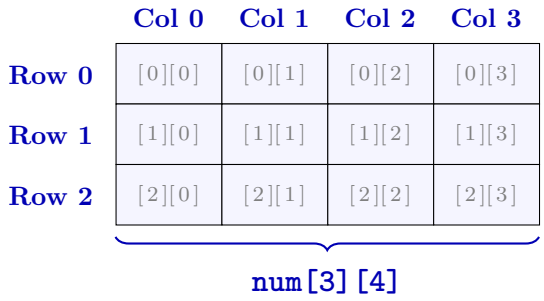
- `Row-size` — the number of rows.
- `Col-size` — the number of columns.
- The total number of elements is `Row-size` × `Col-size`.

Examples.

```
int a[10][10];    // 10 rows x 10 columns (100 elements)
int num[3][4];    //  3 rows x  4 columns ( 12 elements)
```

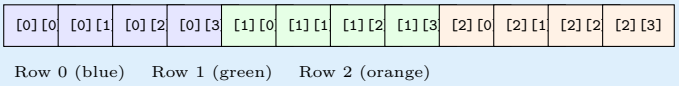
Visualising a 2D Array

The diagram below shows the layout of `int num[3][4]`. Rows are numbered 0–2 and columns 0–3.



Memory Layout

Although we visualise a 2D array as a grid, in memory the elements are stored in *row-major order*: all elements of row 0 come first, followed by all elements of row 1, and so on.



4.2 Initialising 2D Arrays

A two-dimensional array can be initialised at the point of declaration using nested brace-enclosed lists—one inner list per row:

```
int a[2][3] = {
    {1, 2, 3},    // row 0
    {4, 5, 6}     // row 1
};
```

Resulting Layout

The table below shows the values stored in each position after the initialisation above.

	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6

Row Size Can Be Omitted — Column Size Cannot

When an initialiser list is provided, the compiler can deduce the number of rows, so you may write `int a[][3] = {{1,2,3},{4,5,6}};`. However, the *column* size

must always be specified explicitly.

4.3 Reading, Writing, and Processing 2D Arrays

Processing a 2D array requires *nested loops*: the outer loop iterates over the rows, and the inner loop iterates over the columns (or vice versa).

4.3.1 Example 1: Read and Print a 3×5 Array

Problem. Read a 3×5 array of integers from the user, then print all elements.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3][5];

    // Reading
    cout << "Enter 15 integers (3 rows x 5 cols):"
         << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 5; j++) {
            cin >> arr[i][j];
        }
    }

    // Printing
    cout << "Array elements:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 5; j++) {
            cout << arr[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Sample run:

```
Enter 15 integers (3 rows x 5 cols):
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Array elements:
1       2       3       4       5
```

```
6       7       8       9       10
11      12      13      14      15
```

4.3.2 Example 2: Read, Sum, and Print a 4×4 Array

Problem. Read a 4×4 array, compute the summation of all elements, and print the array together with the total.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[4][4];
    int sum = 0;

    cout << "Enter 16 integers (4 x 4):" << endl;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            cin >> arr[i][j];
        }
    }

    // Print and accumulate
    cout << "Array elements:" << endl;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            cout << arr[i][j] << "\t";
            sum += arr[i][j];
        }
        cout << endl;
    }

    cout << "Summation of all elements = "
         << sum << endl;

    return 0;
}
```

Sample run:

```
Enter 16 integers (4 x 4):
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Array elements:
1       2       3       4
5       6       7       8
9       10      11      12
13      14      15      16
```

```
Summation of all elements = 136
```

4.3.3 Example 3: Summation of Each Row

Problem. Read a 3×4 array and compute the summation of each row separately.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3][4];

    cout << "Enter 12 integers (3 x 4):" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            cin >> arr[i][j];
        }
    }

    for (int i = 0; i < 3; i++) {
        int rowSum = 0;
        for (int j = 0; j < 4; j++) {
            rowSum += arr[i][j];
        }
        cout << "Sum of row " << i
             << " = " << rowSum << endl;
    }

    return 0;
}
```

Sample run:

```
Enter 12 integers (3 x 4):
1 2 3 4 5 6 7 8 9 10 11 12
Sum of row 0 = 10
Sum of row 1 = 26
Sum of row 2 = 42
```

Row Summation Pattern

To sum each row independently, reset the accumulator variable (`rowSum`) to zero at the *beginning* of each iteration of the outer loop. This ensures that each row's total is computed from scratch.

4.3.4 Example 4: Replace a Specific Value

Problem. Read a 3×4 array and replace every occurrence of the value 5 with 0.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[3][4];

    cout << "Enter 12 integers (3 x 4):" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            cin >> arr[i][j];
        }
    }

    // Replace 5 with 0
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            if (arr[i][j] == 5) {
                arr[i][j] = 0;
            }
        }
    }

    // Print the modified array
    cout << "Array after replacement:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            cout << arr[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Sample run:

```
Enter 12 integers (3 x 4):
1 5 3 4 5 6 5 8 9 10 5 12
Array after replacement:
1       0       3       4
0       6       0       8
9       10      0       12
```

4.3.5 Example 5: Adding Two 2D Arrays

Problem. Read two 3×4 arrays and compute their element-wise sum into a third array.

```cpp
#include <iostream>
using namespace std;

int main() {
    int a[3][4], b[3][4], c[3][4];

    cout << "Enter first array (3 x 4):" << endl;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 4; j++)
            cin >> a[i][j];

    cout << "Enter second array (3 x 4):" << endl;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 4; j++)
            cin >> b[i][j];

    // Element-wise addition
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 4; j++)
            c[i][j] = a[i][j] + b[i][j];

    // Print the result
    cout << "Sum of the two arrays:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            cout << c[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Sample run:

```
Enter first array (3 x 4):
1 2 3 4 5 6 7 8 9 10 11 12
Enter second array (3 x 4):
10 20 30 40 50 60 70 80 90 100 110 120
Sum of the two arrays:
11      22      33      44
55      66      77      88
99      110     121     132
```

4.3.6 Example 6: Converting a 2D Array to a 1D Array

Problem. Given a 3×4 array, copy all of its elements into a one-dimensional array of size 12.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr2D[3][4];
    int arr1D[12];

    cout << "Enter 12 integers (3 x 4):" << endl;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 4; j++)
            cin >> arr2D[i][j];

    // Flatten 2D -> 1D
    int k = 0;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            arr1D[k] = arr2D[i][j];
            k++;
        }
    }

    // Print the 1D array
    cout << "1D array: ";
    for (int i = 0; i < 12; i++) {
        cout << arr1D[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Sample run:

```
Enter 12 integers (3 x 4):
1 2 3 4 5 6 7 8 9 10 11 12
1D array: 1 2 3 4 5 6 7 8 9 10 11 12
```

Flattening a 2D Array

Converting a 2D array with R rows and C columns into a 1D array of size $R \times C$ is called **flattening**. The element at position `[i][j]` in the 2D array maps to index

$i \times C + j$ in the 1D array. Equivalently, you can use a simple counter variable `k` that increments with every element, as shown above.

4.4 Matrix Diagonal Operations

When a 2D array represents a square matrix (same number of rows and columns), several special index relationships identify important regions of the matrix.

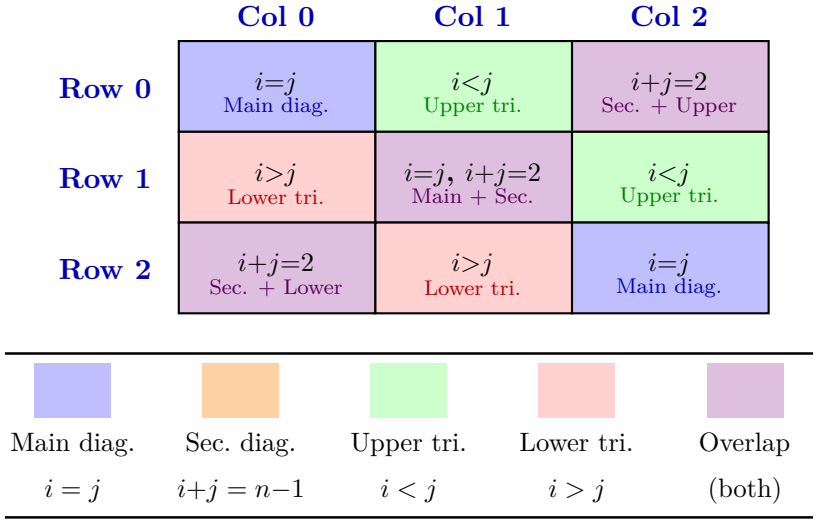
Definition: Diagonal and Triangular Regions

For an $n \times n$ square matrix with row index i and column index j (both starting from 0):

- **Main diagonal:** elements where $i = j$.
- **Secondary diagonal:** elements where $i + j = n - 1$.
- **Upper triangle:** elements where $i < j$ (above the main diagonal).
- **Lower triangle:** elements where $i > j$ (below the main diagonal).

Visual Reference — 3×3 Matrix

The following colour-coded diagram illustrates these regions for a 3×3 matrix. Each cell shows the index condition it satisfies.



The following summary table also maps every cell of a 3×3 matrix to its region:

	Col 0	Col 1	Col 2
Row 0	a_{00} main	a_{01} upper	a_{02} sec., upper
Row 1	a_{10} lower	a_{11} main, sec.	a_{12} upper
Row 2	a_{20} sec., lower	a_{21} lower	a_{22} main

4.4.1 Example 7: Replace Main Diagonal with Zero

Problem. Read a 4×4 matrix and replace all elements on the main diagonal with zero, then print the result.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[4][4];

    cout << "Enter 16 integers (4 x 4):" << endl;
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            cin >> arr[i][j];

    // Replace main diagonal elements with 0
    for (int i = 0; i < 4; i++) {
        arr[i][i] = 0;    // main diagonal: i == j
    }

    // Print the modified matrix
    cout << "Matrix after replacing diagonal:"
         << endl;
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            cout << arr[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Sample run:

```
Enter 16 integers (4 x 4):
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Matrix after replacing diagonal:
0       2       3       4
```

```
5       0       7       8
9       10      0       12
13      14      15      0
```

Accessing the Main Diagonal Efficiently

Since main-diagonal elements satisfy $i = j$, you only need a *single* loop—not nested loops—to visit them all: `for (int i = 0; i < n; i++) arr[i][i] = 0;`.

4.4.2 Example 8: Square Root of Array Elements

Problem. Read a 3×3 array of `double` values and print the square root of each element.

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double arr[3][3];

    cout << "Enter 9 values (3 x 3):" << endl;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            cin >> arr[i][j];

    cout << "Square roots:" << endl;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            cout << sqrt(arr[i][j]) << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Sample run:

```
Enter 9 values (3 x 3):
4 9 16 25 36 49 64 81 100
Square roots:
2       3       4
5       6       7
8       9       10
```

Negative Values under `sqrt`

The `sqrt` function from `<cmath>` returns `NaN` (Not a Number) when given a negative argument. In a robust program you should validate that each value is non-negative before computing its square root.

4.4.3 Example 9: Sum of Main and Secondary Diagonals

Problem. Read a 4×4 integer matrix and compute:

1. The sum of the *main diagonal* elements.
2. The sum of the *secondary diagonal* elements.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[4][4];
    int mainSum = 0, secSum = 0;
    int n = 4;

    cout << "Enter 16 integers (4 x 4):" << endl;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> arr[i][j];

    for (int i = 0; i < n; i++) {
        mainSum += arr[i][i];
        secSum  += arr[i][n - 1 - i];
    }

    cout << "Sum of main diagonal      = "
         << mainSum << endl;
    cout << "Sum of secondary diagonal  = "
         << secSum  << endl;

    return 0;
}
```

Sample run:

```
Enter 16 integers (4 x 4):
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Sum of main diagonal      = 34
Sum of secondary diagonal  = 34
```

Diagonal Summation in a Single Loop

Both diagonal sums can be computed in a single loop of n iterations.

- For the main diagonal, the column index equals the row index: use `arr[i][i]`.
- For the secondary diagonal, the column index is `n - 1 - i`: use `arr[i][n-1-i]`.

Note: in a matrix with an odd dimension, the centre element belongs to *both* diagonals. If you need the combined sum without double-counting, subtract that centre element once.

CHAPTER 5

Strings in C++

5.1 Introduction to Strings

Definition: C-Style Strings

In C++ strings of characters are implemented as an array of characters with a **null terminator** `'\0'`. The null character marks the end of the string so that library functions know where the string data stops.

The general form for declaring a character string is:

```
char String_name[size];
```

where `size` specifies the maximum number of characters the array can hold, *including* the null terminator.

5.1.1 Initialising Strings

A string can be initialised at declaration time in several ways:

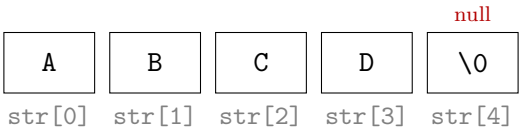
```
char name[11] = "Mazin Alaa";    // size must include '\0'
char str[]    = "ABCD";          // compiler sets size = 5
```

Size Must Include the Null Terminator

When you specify the size explicitly, remember to count the `'\0'` character. For example, `"Mazin Alaa"` has 10 visible characters, so the array must be at least `[11]`.

5.1.2 Character-by-Character Storage

When the compiler stores `str[] = "ABCD"`, the memory layout is as follows:



The Null Terminator

The character `'\0'` (ASCII value 0) is automatically appended by the compiler when you use a string literal. Without it, functions such as `cout`, `strlen`, and `strcpy` will read past the intended boundary, causing undefined behaviour.

5.2 Reading, Writing, and Processing Strings

5.2.1 Example 1 — Printing a String

The following program prints an entire string and then prints it character by character using a loop:

```
#include <iostream>
using namespace std;

int main() {
    char str[] = "Hello, World!";

    // Print the whole string at once
    cout << "Full string: " << str << endl;

    // Print character by character
    cout << "Char by char: ";
    for (int i = 0; str[i] != '\0'; i++) {
        cout << str[i];
    }
    cout << endl;

    return 0;
}
```

Sample run:

```
Full string: Hello, World!
Char by char: Hello, World!
```

Iterating Over a C-String

The idiomatic way to walk through a C-string is to loop while `str[i] != '\0'`. This works because every properly formed C-string is guaranteed to end with the null character.

5.2.2 Example 2 — Converting Lowercase to Uppercase

The library function `toupper()` (declared in `<cctype>`) converts a single lowercase letter to its uppercase equivalent. Non-lowercase characters are returned unchanged.

```
#include <iostream>
#include <cctype>
using namespace std;

int main() {
    char str[50];

    cout << "Enter a string: ";
    cin.getline(str, 50);

    for (int i = 0; str[i] != '\0'; i++) {
        str[i] = toupper(str[i]);
    }

    cout << "Uppercase: " << str << endl;

    return 0;
}
```

Sample run:

```
Enter a string: Hello World
Uppercase: HELLO WORLD
```

5.3 String I/O Functions

Useful String I/O Functions

`cin.getline(str, n);`	Read up to `n-1` characters (including spaces) into `str`.
`cin.get(ch);`	Read a single character (including whitespace) into `ch`.
`cin.ignore(n, delim);`	Discard up to `n` characters or until `delim` is found.
`cin.putback(ch);`	Push character `ch` back into the input buffer.
`cout.put(ch);`	Write a single character to standard output.

```cpp
#include <iostream>
using namespace std;

int main() {
    char line[80], ch;

    cout << "Enter a line: ";
    cin.getline(line, 80);        // reads full line with
       spaces
    cout << "You entered: " << line << endl;

    cout << "Enter a character: ";
    cin.get(ch);                  // reads one character
    cout << "Character: ";
    cout.put(ch);                 // writes one character
    cout << endl;

    cin.ignore(80, '\n');         // flush remaining input

    return 0;
}
```

Sample run:

```
Enter a line: Programming is fun
You entered: Programming is fun
Enter a character: X
Character: X
```

cin » vs. cin.getline()

The extraction operator » stops reading at the first whitespace character, so it cannot read a full sentence. Use `cin.getline()` when you need to capture an entire line including spaces.

5.4 String Library Functions (`string.h`)

The header `<cstring>` (or the older `<string.h>`) provides a rich set of functions for manipulating C-style strings. Table 5.1 summarises the most commonly used ones.

Table 5.1: Common `<cstring>` library functions.

Function	Description	Example

`strlen(s)`	Returns the number of characters in `s`, *excluding* the null terminator.	`char a[]="abcd";`
		`cout « strlen(a);`
		Output: `4`

`strcpy(dest, src)`	Copies the string `src` (including `'\0'`) into `dest`.	`char a[]="abcd", b[10];`
		`strcpy(b, a);`
		b is now `"abcd"`

`strcat(s1, s2)`	Appends a copy of `s2` to the end of `s1`. `s1` must have enough room.	`char a[20]="abcd";`
		`char b[]="1234";`
		`strcat(a, b);`
		a is now `"abcd1234"`

`strcmp(s1, s2)`	Compares two strings lexicographically.
	Returns `0` if equal, a positive value if `s1 > s2`, a negative value if `s1 < s2`.
		`strcmp("abc","abc");` → `0`
		`strcmp("b","a");` → `+`
		`strcmp("a","b");` → `-`

```cpp
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char s1[30] = "Hello";
    char s2[]   = " World";

    cout << "Length of s1: " << strlen(s1) << endl;   // 5
```

```
    strcat(s1, s2);
    cout << "After strcat: " << s1 << endl;            //
       Hello World

    char s3[30];
    strcpy(s3, s1);
    cout << "Copied s3:    " << s3 << endl;            //
       Hello World

    if (strcmp(s1, s3) == 0)
        cout << "s1 and s3 are equal." << endl;

    return 0;
}
```

Sample run:

```
Length of s1: 5
After strcat: Hello World
Copied s3:    Hello World
s1 and s3 are equal.
```

Buffer Overflow with C-Strings

C-style string functions such as `strcpy()` and `strcat()` do *not* check whether the destination array is large enough. Writing past the end of the array causes a **buffer overflow**, which can corrupt other variables, crash the program, or create security vulnerabilities. Always ensure the destination has enough room, including the null terminator.

5.5 `stdlib` Library Functions

The header `<cstdlib>` provides functions for converting between strings and numeric types. Table 5.2 lists the most important ones.

`itoa` is Non-Standard

The function `itoa()` is not part of the C++ standard; it is a compiler extension available in some implementations (e.g. older Borland and MSVC compilers). For portable code, prefer `sprintf()` or `std::to_string()` (C++11).

Table 5.2: Common `<cstdlib>` conversion functions.

Function	Description	Example

`atoi(s)`	Converts the string `s` to an `int`.	`char a[]="1234";`
		`int i = atoi(a);`
		i is now `1234`

`atof(s)`	Converts the string `s` to a `double`.	`char a[]="12.34";`
		`double f = atof(a);`
		f is now `12.34`

`itoa(i, s, base)`	Converts the integer `i` to a string stored in `s` using the specified `base` (e.g. 10 for decimal).	`int i = 1234;`
		`char a[10];`
		`itoa(i, a, 10);`
		a is now `"1234"`

```cpp
#include <iostream>
#include <cstdlib>
using namespace std;

int main() {
    // String to integer
    char num_str[] = "2048";
    int num = atoi(num_str);
    cout << "Integer: " << num << endl;          // 2048

    // String to floating point
    char flt_str[] = "3.14159";
    double pi = atof(flt_str);
    cout << "Double:  " << pi << endl;           // 3.14159

    // Integer to string (non-standard)
    // int val = 255;
    // char buf[20];
    // itoa(val, buf, 10);
    // cout << "String:  " << buf << endl;       // 255

    return 0;
}
```

Sample run:

```
Integer: 2048
Double:  3.14159
```

Missing Null Terminator

If you build a string character by character (e.g. inside a loop), you *must* append `'\0'` manually after the last character. Forgetting it means that every function expecting a C-string will read beyond the intended data, leading to garbage output or undefined behaviour.

C-Strings vs. `std::string`

Modern C++ provides the `std::string` class (from `<string>`) which handles memory management automatically and supports intuitive operators such as `+` for concatenation and `==` for comparison. However, understanding C-style strings is essential because they appear in legacy code, system-level programming, and many library interfaces.

CHAPTER 6

Structures

6.1 Introduction to Structures

Definition: Structure

A **structure** groups several data items together to form a single entity using the `struct` keyword. Unlike arrays, the members of a structure can be of *different* data types.

The general form for defining a structure is:

```
struct struct_name {
    data_type member1;
    data_type member2;
    // ...
    data_type memberN;
};
```

Semicolon After the Closing Brace

The closing brace of a `struct` definition *must* be followed by a semicolon. Omitting it is one of the most common syntax errors when working with structures.

6.2 Declaring Structures

There are three common methods for declaring structure variables in C++.

6.2.1 Method A — Define First, Declare Later

Define the structure type and then declare variables of that type inside a function:

```c
struct data {
    char *name;
    int age;
};

int main() {
    struct data student;       // declare variable of type
       'data'
    student.name = "Ahmed";
    student.age  = 20;
    return 0;
}
```

6.2.2 Method B — Define and Declare Together

Declare one or more variables at the same time as the structure definition:

```c
struct data {
    char *name;
    int age;
} student;                     // 'student' is declared
   immediately

int main() {
    student.name = "Ahmed";
    student.age  = 20;
    return 0;
}
```

6.2.3 Method C — Using `typedef`

The `typedef` keyword creates an alias for the structure type, allowing variables to be declared without the `struct` keyword:

```c
typedef struct {
    char *name;
    int age;
} Student;                     // 'Student' is now a type
   alias

int main() {
    Student s1, s2;            // no 'struct' keyword needed
    s1.name = "Ahmed";
    s1.age  = 20;
```

```
    return 0;
}
```

The Dot Operator

Structure members are accessed using the **dot operator** (`.`). The syntax is:

`variable_name.member_name`

For example: `student.name = "Ahmed";` and `student.age = 20;`

typedef and Multiple Names

With `typedef` you can create multiple convenient aliases for the same structure. This is especially useful when porting code across different projects or when you want shorter type names.

6.3 Practical Examples

6.3.1 Example 1 — Parts Inventory

A structure to represent a single part in an inventory system:

```
#include <iostream>
using namespace std;

struct part {
    int   model_no;
    int   part_no;
    float cost;
};

int main() {
    struct part p1;

    cout << "Enter model number: ";
    cin  >> p1.model_no;
    cout << "Enter part number:  ";
    cin  >> p1.part_no;
    cout << "Enter cost:         ";
    cin  >> p1.cost;
```

```
    cout << "\n--- Part Details ---" << endl;
    cout << "Model No : " << p1.model_no << endl;
    cout << "Part No  : " << p1.part_no  << endl;
    cout << "Cost     : " << p1.cost     << endl;

    return 0;
}
```

Sample run:

```
Enter model number: 101
Enter part number:  5042
Enter cost:         29.95

--- Part Details ---
Model No : 101
Part No  : 5042
Cost     : 29.95
```

6.3.2 Example 2 — Adding Distances (English System)

In the English measurement system, distances are expressed in feet and inches. The following program adds two distances:

```
#include <iostream>
using namespace std;

struct distance {
    int   feet;
    float inches;
};

int main() {
    distance d1, d2, d3;

    // Input first distance
    cout << "Enter feet for d1: ";   cin >> d1.feet;
    cout << "Enter inches for d1: "; cin >> d1.inches;

    // Input second distance
    cout << "Enter feet for d2: ";   cin >> d2.feet;
    cout << "Enter inches for d2: "; cin >> d2.inches;

    // Add the two distances
    d3.inches = d1.inches + d2.inches;
    d3.feet   = d1.feet   + d2.feet;
```

```
    // Handle carry: 12 inches = 1 foot
    if (d3.inches >= 12.0) {
        d3.inches -= 12.0;
        d3.feet   += 1;
    }

    cout << "\nSum = " << d3.feet << " feet, "
         << d3.inches << " inches" << endl;

    return 0;
}
```

Sample run:

```
Enter feet for d1: 5
Enter inches for d1: 8.5
Enter feet for d2: 3
Enter inches for d2: 7.2

Sum = 9 feet, 3.7 inches
```

6.4 Structures within Structures

Structures can be *nested*: a member of one structure can itself be another structure. This is useful for modelling real-world entities that are composed of sub-entities.

6.4.1 Example 3 — Room Area with Nested Distances

```
#include <iostream>
using namespace std;

struct distance {
    int   feet;
    float inches;
};

struct room {
    distance length;
    distance width;
};

int main() {
    room dining;
```

```
    // Input length
    cout << "Enter length (feet): ";
    cin  >> dining.length.feet;
    cout << "Enter length (inches): ";
    cin  >> dining.length.inches;

    // Input width
    cout << "Enter width (feet): ";
    cin  >> dining.width.feet;
    cout << "Enter width (inches): ";
    cin  >> dining.width.inches;

    // Convert to total inches for area calculation
    float len = dining.length.feet * 12 +
       dining.length.inches;
    float wid = dining.width.feet  * 12 +
       dining.width.inches;

    float area_sqin = len * wid;
    float area_sqft = area_sqin / 144.0;  // 144 sq inches
       = 1 sq foot

    cout << "\nRoom area = " << area_sqft
         << " square feet" << endl;

    return 0;
}
```

Sample run:

```
Enter length (feet): 12
Enter length (inches): 6.5
Enter width (feet): 10
Enter width (inches): 3.0

Room area = 128.854 square feet
```

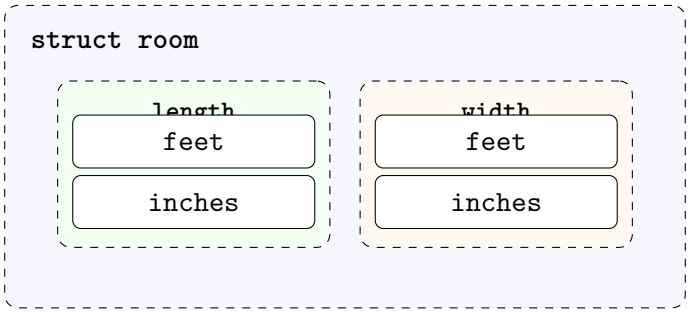
Accessing Nested Members

When structures are nested, you chain the dot operator to reach inner members:

`dining.length.feet` `dining.length.inches` `dining.width.feet` `dining.width.inches`

Each dot moves one level deeper into the structure hierarchy.

The nesting relationship can be visualised as follows:



6.5 Array of Structures

Definition: Array of Structures

Since a `struct` defines a new data type, you can create an **array of structures** just as you would create an array of `int` or `float`. Each element of the array is a complete structure with all its members.

The conceptual layout of an array of structures is shown below:

	name	age
arr[0]	"Ali"	20
arr[1]	"Sara"	22
arr[2]	"Omar"	19
	⋮	⋮
arr[n-1]	"Noor"	21

6.5.1 Example 4 — Array of Student Structures

```
#include <iostream>
#include <cstring>
using namespace std;

struct student {
    char name[30];
    int  age;
};

int main() {
    const int SIZE = 3;
    student arr[SIZE];
```

```
    // Input
    for (int i = 0; i < SIZE; i++) {
        cout << "Enter name of student " << i + 1 << ": ";
        cin.getline(arr[i].name, 30);
        cout << "Enter age of student " << i + 1 << ":  ";
        cin  >> arr[i].age;
        cin.ignore(80, '\n');    // flush newline
    }

    // Output
    cout << "\n--- Student List ---" << endl;
    for (int i = 0; i < SIZE; i++) {
        cout << "Student " << i + 1 << ": "
             << arr[i].name << ", Age: "
             << arr[i].age  << endl;
    }

    return 0;
}
```

Sample run:

```
Enter name of student 1: Ali Hassan
Enter age of student 1:  20
Enter name of student 2: Sara Karim
Enter age of student 2:  22
Enter name of student 3: Omar Faris
Enter age of student 3:  19

--- Student List ---
Student 1: Ali Hassan, Age: 20
Student 2: Sara Karim, Age: 22
Student 3: Omar Faris, Age: 19
```

Accessing Members of Array Elements

Use the array subscript followed by the dot operator:

`arr[i].name` `arr[i].age`

This first selects element `i` from the array and then accesses the desired member of that element's structure.

Structures vs. Arrays

Array	Structure
All elements must be of the *same* data type.	Members can be of *different* data types.
Elements are accessed by **index** (e.g. `a[0]`).	Members are accessed by **name** (e.g. `s.age`).
Useful for storing a collection of *similar* items (e.g. 100 marks).	Useful for grouping *related but different* attributes (e.g. name, age, GPA).
Cannot be assigned directly with `=` in C-style arrays.	Can be assigned directly: `s2 = s1;` copies all members.
Fixed-size, determined at compile time.	Each variable holds one complete set of members.

CHAPTER A

Work Sheets

A.1 Work Sheet 5 — Functions

Instructions

For each question below, write a complete C++ program. Where a *function* is requested, implement the logic inside a user-defined function and call it from `main()`.

Q1. Write a program that reads 20 characters and uses a function to count and return how many of them are **uppercase** letters.

Q2. Write a function that receives a distance expressed in **feet and inches** and converts it to **meters**.
Hint: 1 foot = 12 inches, 1 inch = 2.54 cm.

Q3. Write a function that receives a total number of **seconds** and converts it to the format `hr:mn:sec`. For example, 3661 seconds → `1:1:1`.

Q4. Write a function that takes an integer and determines whether it is **odd** or **even**.

Q5. Write a function that computes the **permutation** $P(n) = n!$ of a given positive integer n.

Q6. Write a function that receives a student's mark and returns the corresponding **grade point** according to the following scale:

Mark Range	Grade Point
90–100	4
80–89	3
70–79	2
60–69	1
Below 60	0

Q7. Write a program that uses a function to compute and print the first n terms of the **Fibonacci** sequence:
0, 1, 1, 2, 3, 5, 8, 13, ...

Q8. Write a function that receives an integer n and returns its **factorial** ($n!$).

Q9. Using a factorial function, evaluate

$$z = \frac{x! - y!}{(x - y)!}$$

for user-supplied values of x and y (where $x \geq y$).

Q10. Write a function that receives a **year** and determines whether it is a **leap year**.
Rule: A year is a leap year if it is divisible by 4, except for century years which must also be divisible by 400.

Q11. Write a program that reads two integers x and y and uses the library function `pow()` to compute x^y.

Q12. Write a function that receives an integer and returns its **reverse**. For example, $765432 \rightarrow 234567$.

Q13. Write a program that uses functions to read the marks of **8 students**, then compute and display their **sum** and **average**.

Q14. Write a function that receives a character and **converts its case** — uppercase to lowercase or vice versa — and returns the result.

Q15. Write a **recursive** function that computes n^p (the power of n raised to p) without using `pow()`.
Hint: $n^p = n \times n^{p-1}$, with base case $n^0 = 1$.

A.2 Work Sheet 6 — Arrays

Instructions

For each question below, write a complete C++ program. Use arrays as specified; you may use functions where appropriate.

Q1. Write a program that reads an array of n integers and checks whether the array is **sorted in ascending order**.

Q2. Write a program that reads an array of n integers and counts the number of elements that are equal to **zero**.

Q3. Write a program that reads two integer arrays of the same size and produces a third array that is their element-wise **sum**.

Q4. Write a program that reads an integer array and creates a new array in which every element is **multiplied by 2**.

Q5. Write a program that reads the daily temperatures for one week (7 values) into an array, then computes and prints the **average temperature**.

Q6. Write a program that reads two sorted arrays and **merges** them into a single sorted array.

Q7. Write a program that reads a 3×4 matrix and computes the **sum of each column**.

Q8. Write a program that reads a square matrix of order n and computes the **sum of the main diagonal** elements.

Q9. Write a program that reads a square matrix of order n and computes the **sum of the secondary diagonal** elements.

Q10. Write a program that reads a 4×5 matrix and **searches** for a user-supplied value, reporting its position (row and column) if found.

Q11. Write a program that reads a one-dimensional array of 12 elements and stores them into a 3×4 two-dimensional array (**1-D to 2-D conversion**).

Q12. Write a program that reads a character array and **splits** it into two arrays: one containing the letters and the other containing the digits.

Q13. Write a program that reads an integer array and replaces every element that is both **even** and **positive** with the value **10**. Print the array before and after the replacement.

Q14. Write a program that reads a square matrix and replaces every **even** element on the **main diagonal** with 0. Print the matrix before and after.

Q15. Write a program that reads a 4×5 matrix and finds the **minimum element** in the entire matrix, printing its value and position (row and column).

Q16. Write a program that reads a 3×3 matrix and **exchanges** (swaps) two user-specified **rows**. Print the matrix before and after the swap.

Q17. Write a program that reads a 3×3 matrix and **exchanges** (swaps) a user-specified **row** with a user-specified **column**. Print the matrix before and after.

Q18. Write a program that reads an array of n integers and performs a **left rotation** by one position. For example, $\{1, 2, 3, 4, 5\} \rightarrow \{2, 3, 4, 5, 1\}$.

Q19. Write a program that reads an array of n integers and **deletes** the element at a user-specified position, shifting subsequent elements to fill the gap.

Q20. Write a program that reads an array of n integers and sorts it in **ascending** order.

Q21. Write a program that reads an array of n integers and sorts it in **descending** order.

Q22. Write a program that reads an array of n integers and computes the **average of the even** numbers only.

Q23. Write a program that reads a 3×4 matrix A and creates a one-dimensional array B of size 3 such that each element $B[i]$ is the **sum of row** i of A.

Example:

```
A = | 1  2  3  4 |    B = | 10 |
    | 5  6  7  8 |        | 26 |
    | 9 10 11 12 |        | 42 |
```

Q24. Write a program that reads a square matrix of order n and finds the **greatest element on the secondary diagonal**.

Q25. Write a program that creates and prints each of the following arrays (the user supplies the size n):

(a) An array containing the first n **even** numbers: $\{2, 4, 6, 8, \ldots\}$

(b) An array containing the first n **odd** numbers: $\{1, 3, 5, 7, \ldots\}$

(c) An array whose elements follow the pattern $\{1, 2, 4, 8, 16, \ldots\}$ (powers of 2).

A.3 Work Sheet 7 — Strings

Instructions

For each question below, write a complete C++ program using C-style strings (`char` arrays). Make use of the string I/O and library functions discussed in Chapter 5.

Q1. Write a program that reads a string from the user, prints the complete string, and then prints its characters in **reverse order** (one character per line).

Example:

```
Input:  Hello
Output: Hello
        o
        l
        l
        e
        H
```

Q2. Write a program that reads a string and **swaps the case** of every letter — uppercase letters become lowercase and vice versa. Non-letter characters remain unchanged.

Example:

```
Input:  Hello World 123
Output: hELLO wORLD 123
```

Q3. Write a program that reads a full sentence (using `cin.getline()`) and prints each **word on a separate line**. Assume words are separated by single spaces.

Example:

```
Input:  Structured Programming in C++
Output: Structured
        Programming
        in
        C++
```

Q4. Write a program that demonstrates the use of the following **I/O functions**:

- `cin.getline()` — to read a full line,
- `cin.get()` — to read a single character,
- `cin.ignore()` — to discard remaining input,
- `cin.putback()` — to push a character back,
- `cout.put()` — to output a single character.

Display appropriate messages so the output clearly shows which function is being used.

Q5. Write a program that demonstrates the use of the following **string library functions**:

- `strlen()` — to find the length of a string,
- `strcpy()` — to copy one string into another,
- `strcat()` — to concatenate two strings,
- `strcmp()` — to compare two strings.

Read two strings from the user and apply each function, printing the result after every operation.